Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Fetch as your first real promise

Today

Welcome to your next lesson: Fetch as your first real promise.

I'm preparing your personalized lesson based on what you've learned so far, what's up next, and the best way to help you practice.

Step by step, you're moving forward.

Imagine you call a friend and ask them for a favor. 📞

You don't get the favor right away — but the call has started the process.

fetch() is like that in JavaScript: it starts a request to some URL on the internet. The result comes back later.

Let's start super small:

In the code editor, type and run this only:

Javascript
```
fetch("https://jsonplaceholder.typicode.com/posts/1");
```
No console.log yet, nothing else.

Run it, then tell me:

Did you see anything in the console or on the screen, or was it basically "silent"?

Output

Your code ran successfully without any output.

Nice, that "silent success" is exactly what I wanted. ☑

fetch("https://jsonplaceholder.typicode.com/posts/1");

→ starts an API request in the background

→ but doesn't print anything by itself.

Next step: I want you to see what fetch() gives back.

In the editor, change your code to:

Javascript
```
const result = fetch("https://jsonplaceholder.typicode.com/posts/1");
console.log(result);
```
Run this version, then paste (or describe) what you see in the console — especially how result is shown.

Output

Promise {}

Perfect 👌

So now you've seen:

Javascript
```
const result = fetch("https://jsonplaceholder.typicode.com/posts/1");
console.log(result);
// → Promise {}
```
That Promise {} is JavaScript's way of saying:

"Here is a ticket for a value that will be ready later."

You don't have the final data yet — you just have this special "ticket object" that represents the future result.

Quick check-in (no code change yet):

Does this "ticket for a future value" idea make sense so far, or should I rephrase it in another way?

That makes sense

Great 👀

Next concept: how to use that "ticket".

We can tell JavaScript: "When this Promise is done, then run some code."

We do that with .then().

In your editor, replace your code with this:

Javascript
```
const result = fetch("https://jsonplaceholder.typicode.com/posts/1");

result.then(() => {
  console.log("Request finished!");
});
```
First, predict in your head: will "Request finished!" appear immediately, or only after a tiny delay?
Then run the code and tell me what you observed in the console.
should be a delay based on hos long it takees to send and reeie a response from that URL
Output
Request finished!
(pretty much instant)

Your reasoning is spot on 💯

Yes: the delay depends on how long it takes to send the request and get the response

back. On a fast network with a small test API like this, it can feel almost instant, but it’s still:

Start the request with fetch(...)
Some time passes (maybe tiny)
Then the code inside .then() runs: console.log("Request finished!")

So .then() means: “run this later, when the Promise is done.”

Now let’s mix sync + async and look at order. 😤

In the editor, change your code to:

Javascript
```
console.log("A: starting");

const result = fetch("https://jsonplaceholder.typicode.com/posts/1");

result.then(() => {
  console.log("C: request finished");
});

console.log("B: ending sync code");
```
Before you run it, what order do you think will show in the console?

A → B → C
A → C → B

Just tell me which sequence you expect.

a b c, the .then should be delayed

Exactly right 🎯

console.log is synchronous, so:

"A: starting" runs right away.
fetch(...) starts the request, returns a Promise.
"B: ending sync code" runs next (still synchronous).
Later, when the request finishes, the .then callback runs and logs "C: request finished".

So the actual console order will be:

A: starting
B: ending sync code
C: request finished
Go ahead and run that code now and confirm you see A → B → C in the console.

Did the output match that order for you?

yes
Output

```
A: starting
B: ending sync code
C: request finished
```

Nice work, Ron — you just hit all the key pieces of this lesson 🎉

Let me quickly check you really own each idea:

Starting an API request
In your own words: what does fetch("https://jsonplaceholder.typicode.com/posts/1") do, even if we don’t log anything or look at any data?

Promise as a ticket
When you saw Promise {} in the console, how would you explain what that is to a friend who knows a bit of JavaScript but hasn’t seen Promises yet?

Keep answers short and simple — I just want to confirm your understanding before we officially wrap this lesson.

fetch starts the process of sending the request which ill ome back later
Promise is a placeholder to use ot retrieve the outut when its ready, its like a tickt you get a the dry cleaners when dropping off a jacket, you use it when you come to pickup the jacket to assure you receive the correct jacket
Your explanations are excellent — especially the dry-cleaner ticket analogy 😂🧺

Quick recap of what you now know:

fetch(url) starts an API request; the response comes back later.
The value you get from fetch(...) is a Promise — a ticket / placeholder for a result that will be ready in the future.
.then(() => { ... }) lets you run code after that Promise finishes.
With mixed code:
normal console.log runs first (A, B)
the .then log runs later (C), so the order is A → B → C.

You’ve fully mastered this lesson 🎯

From here, the next lessons will go deeper into using that response data and more Promise patterns.

Want to keep going to the next async lesson now, or take a break and come back later?


```
console.log("A: starting");

const result = fetch("https://jsonplaceholder.typicode.com/posts/1");

result.then(() => {
  console.log("C: request finished");
});

console.log("B: ending sync code");
```